Chapter 1 Reloaded: Dual-Core Light Tasking – Blinking LEDs on Both Cores

March 13, 2026 · Updated: April 10, 2026

Link	Purpose
C Tutorial Chapter 1 – LED (Important)	Freenove's official C version – Chapter 1 introduction to LED control
C Tutorial Chapter 2 – LED (Important)	Freenove's C Video description
Alire crate	Alire crate containing the Ada code in this chapter
GNATdoc documentation for this chapter	Automatically generated HTML documentation for the Ada code in this chapter

What has changed?
- Alire.toml
- pico_ada_c01_blink_lt.gpr
- double_blink.ads
- double_blink.adb
- double_blink-main.adb

Result
Conclusion

Yesterday I published the first chapter of this tutorial. In it I explained why both Ravenscar and the initial light tasking attempts had failed for me. Within minutes, helpful suggestions arrived in the Ada Telegram group and on the Ada Forum. Even better, a new version of the light tasking runtimes was announced almost immediately.

The Ada community never ceases to impress me.

Thanks to these updates, I could finally implement all three samples from Chapter 1 with proper Ada tasking support. No more manual timer initialisation – at least in the new dual-core version I can now use the proper delay until keyword for accurate, drift-free timing. The older single-core samples still use plain delay, but I've learned the right way to do precise periodic tasks from damaki's excellent pico_smp_demo examples on GitHub. His code was the key inspiration for getting delay until and multi-core tasking working smoothly together. Even more exciting: this runtime supports multi-core execution out of the box, so one core can blink the internal LED while the other happily blinks an external LED at the same time.

So here it is: **Blink Reloaded – now with dual cores**.

This version is a huge improvement. Both LEDs blink simultaneously on separate cores, and I use `Ada.Real_Time` with `delay until` for precise, slippage-free timing. [1]

What has changed?

Alire.toml

Multi-core support requires more dependencies and stricter version pins. A simple `pico_bsp = "^2"` is no longer sufficient – you need the very latest versions. Both `pico_bsp` and `light_tasking_rp2040` provide initialisation code, so you must explicitly choose which startup to use (among other settings).

```
[[depends-on]]
rp2040_hal                          = "^2.7"
pico_bsp                            = "^2.2"
light_tasking_rp2040                = "^15.2"

[configuration.values]
rp2040_hal.Use_Startup              = false
rp2040_hal.Interrupts               = "bb_runtimes"
light_tasking_rp2040.Max_CPUs       = 2
light_tasking_rp2040.Board          = "rpi_pico"
```

Note 1: You may need to run `alr index --update-all` to fetch the newest indices, depending on when your last automatic update occurred. **Note 2:** `light_tasking_rp2040.Board` can be finicky on the first build. If it fails, try removing it, building once, then adding it back.

pico_ada_c01_blink_lt.gpr

You now build your own runtime using `runtime_build.gpr` instead of `pico_bsp.gpr`. Remove `with "pico_bsp.gpr";` and add `with "runtime_build.gpr";`. Update these lines as shown:

```
with "config/pico_ada_c01_blink_lt_config.gpr";
with "runtime_build.gpr";

---
--  Sample Project to make both the internal and an external L
--  different rates using both cores of the RP2040.
--
project Pico_Ada_C01_Blink_LT is

   for Target                use Runtime_Build'Target;
   for Runtime ("Ada")       use Runtime_Build'Runtime ("Ada");
   for Object_Dir            use "obj/" & Pico_Ada_C01_Blink_LT_
…
   Ada_Compiler_Switches := Pico_Ada_C01_Blink_LT_Config.Ada_C
…
   package Linker is
      for Switches ("Ada") use Runtime_Build.Linker_Switches &
   end Linker;
…
end Pico_Ada_C01_Blink_LT;
```

double_blink.ads

This restricted runtime has no heap and limited stack sizes, so all task objects must be declared at library level (top-level). We can no longer keep everything inside a single procedure – a package is required.

In the spec we declare global renames, the blink procedure, and – the highlight – a task explicitly assigned to CPU 2 (the second core).

```
pragma License (Modified_Gpl);
pragma Ada_2022;
pragma Extensions_Allowed (On);

with RP.GPIO;
with Pico;

---
--  Sample to make both the internal and an external LED blink
--
package Double_Blink is

   ---
   --  GPIO points for the internal (on-board) and external LE
   --
   Internal_LED  : RP.GPIO.GPIO_Point renames Pico.GP25;
   External_LED  : RP.GPIO.GPIO_Point renames Pico.GP15;

   ---
   --  Task that runs on the second core of the RP2040. As a s
   --  it is automatically activated at startup.
   --
   task Core_2 with
      CPU => 2
   ;

   ---
   --  Blinks an LED attached to a Raspberry Pi Pico
   --
   --  @param LED      GPIO pin the LED is connected to
   --  @param On_Time  Duration the LED stays on
   --  @param Off_Time Duration the LED stays off
   procedure Blink_LED
      (LED      : in out RP.GPIO.GPIO_Point;
       On_Time  : in     Duration;
       Off_Time : in     Duration) with
      No_Return;

end Double_Blink;
```

And yes – `Blink_LED` really is a **no-return** procedure 😮

double_blink.adb

Here we implement the actual blinking logic. Instead of fixed delays (which accumulate slippage/jitter over time), we track the absolute next toggle time using `Ada.Real_Time`. This keeps the duty cycle precise even after many minutes or hours.

The `Core_2` task body simply calls `Blink_LED` with chosen prime-number-based periods (divided by 1000) so the LEDs desynchronise nicely.

```
pragma License (Modified_Gpl);
pragma Ada_2022;
pragma Extensions_Allowed (On);

with Ada.Real_Time;

package body Double_Blink is
   use type Ada.Real_Time.Time_Span;
   use type Ada.Real_Time.Time;

   procedure Blink_LED
      (LED      : in out RP.GPIO.GPIO_Point;
       On_Time  : in     Duration;
       Off_Time : in     Duration)
   is
      On_Period  : constant Ada.Real_Time.Time_Span := Ada.Rea
      Off_Period : constant Ada.Real_Time.Time_Span := Ada.Rea
      Next       : Ada.Real_Time.Time               := Ada.Rea
   begin
      LED.Configure (RP.GPIO.Output);

      loop
         --  On cycle
         LED.Set;
         Next := Next + On_Period;
         delay until Next;

         --  Off cycle
         LED.Clear;
         Next := Next + Off_Period;
         delay until Next;
      end loop;
   end Blink_LED;

   task body Core_2 is
   begin
      Blink_LED
         (LED      => Internal_LED,
          On_Time  => 1.009,
          Off_Time => 0.503);
   end Core_2;
end Double_Blink;
```

double_blink-main.adb

Core 1 starts the usual way – via a standalone main procedure. It simply calls `Blink_LED` for the external LED.

```
pragma License (Modified_Gpl);
pragma Ada_2022;
pragma Extensions_Allowed (On);

---
--  Main procedure – runs on Core 1 of the RP2040.
--
procedure Double_Blink.Main is
begin
   Blink_LED
      (LED      => External_LED,
       On_Time  => 0.751,
       Off_Time => 0.353);
end Double_Blink.Main;
```

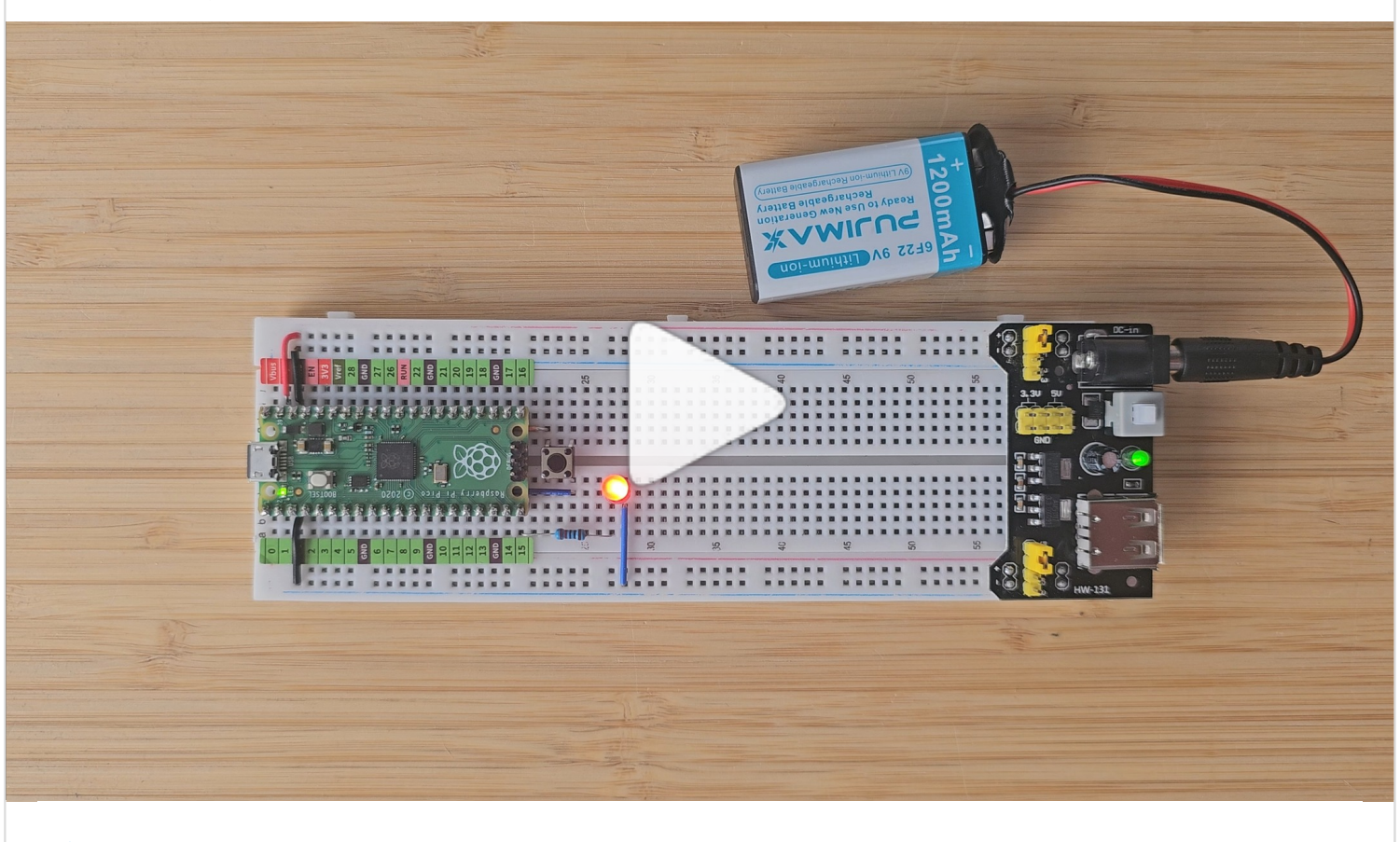
Result

See for yourself:

Pi blinking both LEDs

martin.krischik
Original audio

View profile



View more on Instagram

2 likes

martin.krischik

Raspberry Pi Pico using both of it's cores to blink an LED. Programmed in Ada. Details here:

https://pi-ada-tutorial.sourceforge.io/pico_ada_c01_blink_lt/

Add a comment...

Conclusion

That was great fun – and very Ada-like: „If it compiles (and links), it runs".

Most of the tricky work happened during linking, getting the runtime pieces to cooperate. But there is actually **no real drawback** 😲

I initially worried the extra runtime would bloat the binary size. To my surprise it did the opposite – especially with the `("-Wl,--gc-sections")` linker flag, which removes dead code very effectively. The light real-time runtime seems to optimise far better.

```
> ls -lah */bin/*.uf2
-rw-r--r--  1 martin  _developer    28K 13 Mar 18:44 pico_ada_
-rw-r--r--  1 martin  _developer    31K 13 Mar 18:50 pico_ada_
-rw-r--r--  1 martin  _developer    28K 13 Mar 18:50 pico_ada_
-rw-r--r--  1 martin  _developer    28K 13 Mar 18:50 pico_ada_
-rw-r--r--  1 martin  _developer    64K 13 Mar 18:58 pico_ada_
-rw-r--r--  1 martin  _developer    64K 13 Mar 18:58 pico_ada_
-rw-r--r--  1 martin  _developer    64K 13 Mar 18:58 pico_ada_
```

For comparison, without `"-Wl,--gc-sections"` you get much larger files:

```
> ls -lah */bin/*.uf2
-rw-r--r--  1 martin  _developer    50K 13 Mar 19:06 pico_ada_
-rw-r--r--  1 martin  _developer    52K 13 Mar 19:06 pico_ada_
…
-rw-r--r--  1 martin  _developer   182K 13 Mar 19:06 pico_ada_
…
```

The only real cost is a more complex initial setup and a longer first build (because of `runtime_build.gpr`). But that is a one-time effort. From now on I will stick with this runtime configuration.

Happy double blinking!

---

[1] **Why `delay until` is better for real-time work?**

`delay` is **relative** – if the task is delayed slightly before the statement (due to preemption, interrupts, etc.), the actual wait time grows shorter each cycle. Over many iterations this creates **slippage** (timing drift / cumulative jitter).

`delay until <absolute time>` always wakes up as close as possible to the requested **absolute** moment. By recalculating the next target time *after* each cycle (`Next := Next + Period`), the period stays precise and does not drift – perfect for periodic real-time behaviour like LED blinking, motor control or sampling.

AUTHOR
Martin Krischik

← Chapter 1: Blinking LEDs – Getting Started with Ada on the Raspberry Pi Pico
March 12, 2026

Chapter 2 – Button & LED →
March 20, 2026

© 2026 Martin Krischik
Powered by Hugo & Congo